

# PDC Assignment

## Part 1: N-body Solver

Jay Dally-Sparre / a1950230

### Overview

The goal of this assignment was to take a naive N-body solver written in C with MPI and optimize its memory usage.

### Part 1A Design

The requirements were to replace the timestep call to `MPI_Allgather` with an explicit ring communication pattern. With this in mind, I provide a function which effectively emulates `MPI_Allgather` using this pattern. The idea is,  $p-1$  times, we forward our current block (starting with our own data) to the process with rank one greater than our own (under modulo), receiving into the array block position one rank below us (under modulo). Then, we update our indexes backwards, as we will then receive what the process before us received on the previous iteration, and likewise send what we received on the previous iteration.

The below diagram illustrates this process for the first two iterations. Note this is for  $n = 3$ , with each block<sub>*i*</sub> representing a segment of the array of particle data.

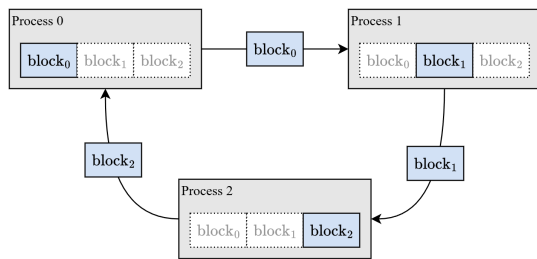


Figure 1.1

*Initial conditions for ring communication.*

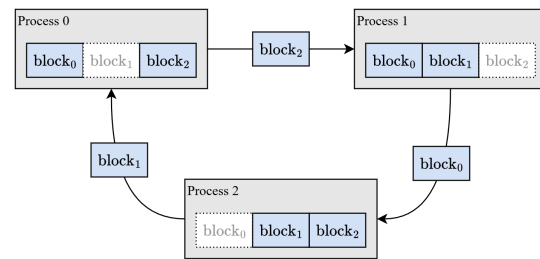


Figure 1.2

*After one iteration.*

I note that I used `MPI_Sendrecv` for the actual point to point communication. This avoids having to do even/odd ordering or any other method, as it behaves as an atomic send/receive. Finally, this approach works for the case of 1 process, as we do this process  $p-1$  times, which for  $p = 1$  means it never runs at all.

### Part 1B

This section encompasses a more substantial refactoring of the solver; the goal being to make all permanent storage of masses, positions, etc. local. Notably, my design features an intertwining of the computation and ring communication to support this. The function `Compute_force` has been refactored to take in internal and external arrays; the idea being that we will compute the forces for a given particle for each block as its passed around the ring, as opposed to all at once. I also added a parameter to indicate whether we are operating on our own buffer; if so, there is a check to avoid particle self-interaction. To support ring communication, two buffers have been created corresponding to position and mass respectively. Naturally, the `Ring_Allgather` function has been replaced with a `Cycle_buffers` function, as we need to do the ring communication gradually rather than all at once.

So the time step loop now does the following: (1) initializes our ring buffers to our own data, (2) computes local forces, (3) cycles through the ring, computing forces with the positions/masses from other processes, (4) updates as before.

The figure below illustrates the buffer cycling. From the start of the time-step loop, both buffers are initialized to their processes local position and mass arrays.

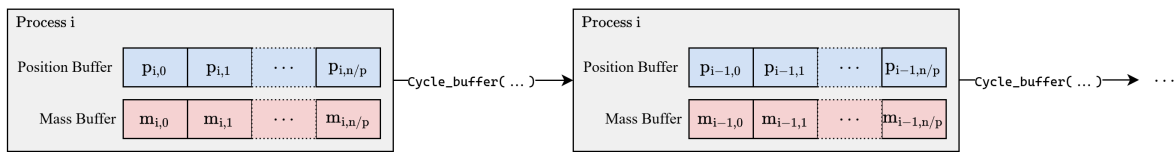


Figure 2.1

*Buffer state after each call to Cycle\_buffer().*

Notably I also replaced `MPI_Sendrecv` with `MPI_Sendrecv_replace`. Conceptually they are similar but `MPI_Sendrecv_replace` supports using the same buffer for send and receive which simplifies the code.

Finally, I use `MPI_Gather` to get the position, mass, and velocity data from each individual process into buffers of size  $n$  on process 0 for the output stage. This still satisfies the requirements as this data is not permanently stored across all processes.

## Validation

For validation, I had the idea to write a bash script to mimic a testing framework. The script loops over the `testcases` folder and for each `.test` file, which contains a predetermined set of arguments to run, it tests against a set of initial conditions supplied in another file. This test is repeated for different  $n$  divisible by the number of particles (otherwise we would have problems with how the N-body solver distributes its data; there's an assumption that all processes receive the same amount). I wrote a quick Python script which uses regular expressions to extract the numbers between two files and compare whether their difference is within some acceptable epsilon tolerance (`diff` checks might be problematic as floating point addition is not associative) and found this effective for testing correctness on my end.

## Design Comparison

The new program design is much more memory efficient than the base implementation; each process now only has to permanently store arrays of size  $n/p$ , where  $n$  is the number of particles, and  $p$  the number of processes. Previously, each process had to store arrays of size  $n$ . This means that for large  $n$  it becomes much more feasible to compute as memory overhead is kept to a minimum.

There are a few trade-offs with this approach. Replacing `MPI_Allgather` with a naive ring may potentially be slower as MPI may use a heavily optimized algorithm to perform the collective communication. For part 1B in particular, it may be noted that now we communicate the mass and the position  $p-1$  times, as opposed to before, when we only communicated the position  $p-1$  times. This means we are communicating more data which should be inherently slower.

Notably, on a larger test case ( $n = 36$ ,  $p = 12$ ) I observed the following results:

```
ref: Elapsed time = 2.965679e-02 seconds
pla: Elapsed time = 3.865023e-01 seconds
plb: Elapsed time = 6.014859e-02 seconds
```

We note that part 1B performs much worse on this than the reference and part 1A. One reason this could be the case is because I do not pack position/masses into a struct prior to sending, which may be slower as we call `MPI_Sendrecv_replace` twice, or otherwise it could be related to the aforementioned extra data which is communicated.